

ICPC JRC Team Codebook

C++ STL, Algorithms & Data Structures Reference

For beginner teams aiming for a Top-5 finish

How to use this book:

- Print it and bring it as your team's physical reference (ICPC allows a printed codebook).
- Every snippet is a self-contained, tested **template** – copy, rename variables, paste into your solution.
- Comments in the code explain *what* and *when*, not just *how*.
- Section 1 covers contest strategy specifically for JRC (Junior/Regional/Divisional style) rounds.
- Section 20 is your cheat sheet for picking the right algorithm under time limits.

Compile all code with: `g++ -std=c++17 -O2 -Wall solution.cpp -o solution`

Practice judge: vjudge.net, codeforces.com, icpc.global

Contents

1	Contest Strategy for JRC Teams	3
1.1	Before the Contest	3
1.2	During the Contest	3
1.3	Common Beginner Mistakes to Avoid	3
2	C++ Contest Template & Fast I/O	3
2.1	I/O Speed Tricks	4
3	C++ STL Essentials	4
3.1	Pair and Tuple	4
3.2	Vector – the dynamic array you’ll use most	4
3.3	Set / Multiset – sorted unique (or non-unique) elements	4
3.4	Map / Unordered Map – key to value lookup	5
3.5	Stack, Queue, Deque, Priority Queue	5
3.6	String Functions	5
3.7	Bitset – compact fixed-size bit array	5
3.8	Other Handy ‘<algorithm>’ Functions	6
4	Sorting, Searching & Two-Pointer Techniques	6
4.1	Custom Sort Comparators	6
4.2	Binary Search	6
4.3	Two Pointers	6
4.4	Sliding Window (fixed and variable size)	7
5	Number Theory	7
5.1	GCD, LCM, Fast Exponentiation	7
5.2	Modular Inverse (needed for modular division)	7
5.3	Sieve of Eratosthenes – all primes up to N	7
5.4	Prime Factorization & Divisors	8
6	Graph Algorithms	8
6.1	Representation & Traversal (BFS / DFS)	8
6.2	Dijkstra – shortest path, weighted, no negative edges	9
6.3	Bellman-Ford – handles negative edges, detects negative cycles	9
6.4	Floyd-Warshall – all-pairs shortest path	9
6.5	Topological Sort (Kahn’s Algorithm, BFS-based)	10
6.6	Disjoint Set Union (DSU / Union-Find)	10
6.7	Minimum Spanning Tree – Kruskal’s Algorithm	10
6.8	Minimum Spanning Tree – Prim’s Algorithm	11
7	Trees & Range Query Structures	11
7.1	Fenwick Tree / Binary Indexed Tree (BIT)	11
7.2	Segment Tree – range sum, point update	11
7.3	Sparse Table – O(1) range minimum query (static array, no updates)	12
7.4	Binary Lifting – LCA (Lowest Common Ancestor) in O(log n) per query	12
8	Dynamic Programming	12
8.1	0/1 Knapsack	12
8.2	Longest Increasing Subsequence (LIS)	13
8.3	Longest Common Subsequence (LCS)	13
8.4	Coin Change (minimum coins to make amount)	13
8.5	DP General Tips	13
9	String Algorithms	14

9.1	KMP (Knuth-Morris-Pratt) – pattern matching in $O(n + m)$	14
9.2	Z-Function – another $O(n)$ string matching / prefix tool	14
9.3	Trie – prefix tree for fast string prefix queries	14
9.4	Polynomial String Hashing – $O(1)$ substring comparison after $O(n)$ precompute	15
10	Basic Computational Geometry	15
11	Bit Manipulation	16
12	More Number Theory: Extended Euclid & Combinatorics	16
12.1	Extended Euclidean Algorithm	16
12.2	Combinatorics – $nCr \bmod p$, Factorials, Pascal's Triangle	16
13	Advanced Graphs	17
13.1	0-1 BFS – shortest path when edge weights are only 0 or 1	17
13.2	Strongly Connected Components – Kosaraju's Algorithm	17
13.3	Segment Tree with Lazy Propagation – range update, range sum query	18
14	Bitmask DP, Digit DP & Matrix Exponentiation	18
14.1	Bitmask DP – Travelling Salesman Problem (TSP)	18
14.2	Digit DP – count numbers in $[0, N]$ satisfying a digit property	19
14.3	Matrix Exponentiation – $O(k^3 \log n)$ for linear recurrences (e.g. Fibonacci)	19
15	Recursion & Backtracking	19
15.1	N-Queens – classic backtracking template	19
15.2	Generating All Subsets / Permutations	20
16	Bipartite Graph Check & Graph Colouring	20
17	STL Container Complexity Summary	21
18	Prefix Sums & Interval Techniques	21
18.1	1D Prefix Sum – $O(1)$ range sum after $O(n)$ precompute	21
18.2	2D Prefix Sum – $O(1)$ rectangle sum queries	21
18.3	Difference Array – $O(1)$ range update, $O(n)$ to rebuild	21
18.4	Merging Overlapping Intervals	21
19	Modular Arithmetic Quick Reference	22
20	Complexity Cheat Sheet	22
20.1	Time Limit Rule of Thumb	22
20.2	Common Algorithm Complexities	22
21	Debugging & Submission Checklist	22
21.1	Wrong Answer	23
21.2	Runtime Error	23
21.3	Time Limit Exceeded	23
21.4	General Testing Habits	23
22	Quick-Reference: Which Data Structure Do I Need?	23
23	4-Week Practice Plan (Appendix)	23
23.1	Problem-Tag Checklist	24
Notes		24

1 Contest Strategy for JRC Teams

1.1 Before the Contest

- Agree on roles: one **coder**, one **reader/tester**, one **whiteboard/math** person. Rotate the keyboard every 45–60 minutes if your team allows it.
- Print this codebook and highlight the 5–6 templates you are most comfortable with.
- Set up your editor/template file (Section 2) before the clock starts, if permitted.
- Decide a naming convention for test files, e.g. `A.cpp`, `B.cpp`.

1.2 During the Contest

- **Read all problems first** (10–15 min). Sort by estimated difficulty using statement length, I/O size, and sample complexity.
- Solve **easiest problems first** to build up accepted count and morale – ICPC ranks by (most solved, then least penalty time).
- One person codes while another dry-runs the sample input *by hand* before submitting.
- If a submission gets Wrong Answer, do not resubmit blindly – recheck edge cases: $n = 1$, empty input, all-equal values, overflow, negative numbers.
- Keep a shared queue on the whiteboard: problem letter, status (Reading / Coding / Testing / AC / Stuck), owner.
- Time-box each problem. If stuck for 20+ minutes, swap to a different problem and return later.

1.3 Common Beginner Mistakes to Avoid

- Using `int` where the answer can exceed $2^{31} - 1 \approx 2.1 \times 10^9$ – use `long long`.
- Forgetting to reset global arrays/adjacency lists between test cases in multi-test-case problems.
- Off-by-one errors in loops – always double check whether the problem is 0-indexed or 1-indexed.
- Using `endl` everywhere (flushes the buffer, slows output) instead of `"\n"`.
- Not enabling fast I/O (Section 2) – default `cin/cout` can be too slow for 10^5+ inputs.
- Comparing floating point numbers with `==` instead of an epsilon tolerance.

2 C++ Contest Template & Fast I/O

Paste this at the top of every solution. It gives you fast I/O, common type aliases, and useful macros.

```

1 #include <bits/stdc++.h>    // Includes almost the entire STL -- fine for contests
2 using namespace std;
3
4 // ---- Type aliases (save typing, avoid overflow bugs) ----
5 typedef long long ll;      // use ll instead of int for anything that could be large
6 typedef unsigned long long ull;
7 typedef pair<int,int> pii;
8 typedef pair<ll,ll> pll;
9 typedef vector<int> vi;
10 typedef vector<ll> vll;
11 typedef vector<pii> vpii;
12
13 // ---- Useful macros ----
14 #define all(x) (x).begin(), (x).end()    // range-based sort(all(v)) etc.
15 #define rall(x) (x).rbegin(), (x).rend()
16 #define pb push_back
17 #define sz(x) (int)(x).size()
18 #define fi first
19 #define se second
20
21 const int MOD = 1e9 + 7;    // common competitive-programming modulus
22 const ll INF = 4e18;       // "infinity" for ll (won't overflow when added twice)
23 const int IINF = 1e9;      // "infinity" for int
24
25 int main(){
26     // Fast I/O: MUST be called before any cin/cout usage
27     ios_base::sync_with_stdio(false);
28     cin.tie(NULL);
29
30     int t = 1;
31     // cin >> t;           // uncomment if the problem has multiple test cases
32     while (t--) {
33         // ---- solve() logic goes here ----

```

```

34     int n;
35     cin >> n;
36     cout << n << "\n";           // use "\n" not endl (endl flushes stdout, slow)
37 }
38 return 0;
39 }

```

2.1 I/O Speed Tricks

```

1 // 1) Always pair sync_with_stdio(false) + cin.tie(NULL) at the start of main().
2 // 2) Prefer "\n" over endl in tight loops.
3 // 3) For reading a whole line (with spaces):
4 string line;
5 getline(cin, line);
6
7 // 4) printf/scanf can be mixed with cin/cout ONLY if you also call:
8 //     ios_base::sync_with_stdio(true); -- otherwise avoid mixing them.
9
10 // 5) Reading until EOF (unknown number of inputs):
11 int x;
12 while (cin >> x) {
13     // process x
14 }

```

3 C++ STL Essentials

3.1 Pair and Tuple

```

1 pair<int,int> p = {3, 5};           // p.first = 3, p.second = 5
2 p = make_pair(1, 2);
3 tuple<int,int,int> t = {1, 2, 3};
4 int a, b, c;
5 tie(a, b, c) = t;                  // unpack tuple
6 auto [x, y] = p;                   // C++17 structured bindings (very handy!)

```

3.2 Vector – the dynamic array you’ll use most

```

1 vector<int> v;                      // empty vector
2 vector<int> v2(10);                 // 10 elements, all 0
3 vector<int> v3(10, -1);             // 10 elements, all -1
4 vector<vector<int>> grid(n, vector<int>(m, 0)); // n x m 2D grid initialised to 0
5
6 v.push_back(5);                    // add to end -- O(1) amortised
7 v.pop_back();                      // remove last -- O(1)
8 v.size();                          // number of elements
9 v.empty();                         // true if size == 0
10 v.clear();                         // remove all elements
11 v.insert(v.begin() + 2, 99);       // insert 99 at index 2 -- O(n)
12 v.erase(v.begin() + 2);            // remove element at index 2 -- O(n)
13 sort(v.begin(), v.end());          // ascending sort -- O(n log n)
14 sort(v.rbegin(), v.rend());        // descending sort
15 reverse(v.begin(), v.end());       // reverse in place
16 int mx = *max_element(v.begin(), v.end());
17 int mn = *min_element(v.begin(), v.end());
18 ll sum = accumulate(v.begin(), v.end(), 0LL); // 0LL avoids overflow

```

3.3 Set / Multiset – sorted unique (or non-unique) elements

```

1 set<int> s;                         // sorted, unique, backed by a balanced BST
2 s.insert(5);                       // O(log n)
3 s.erase(5);                       // O(log n)
4 s.count(5);                        // 0 or 1 -- is 5 present?
5 auto it = s.find(5);               // iterator to 5, or s.end() if absent
6 if (!s.empty()) {
7     int smallest = *s.begin();
8     int largest  = *s.rbegin();
9 }

```

```

10 // lower_bound(x): first element >= x.  upper_bound(x): first element > x.
11 auto lb = s.lower_bound(5);
12 auto ub = s.upper_bound(5);
13
14 multiset<int> ms;                // like set but allows duplicates
15 ms.insert(3); ms.insert(3);     // now contains two 3's
16 ms.erase(ms.find(3));          // erase ONE occurrence (ms.erase(3) removes ALL of them
    !)
```

3.4 Map / Unordered Map – key to value lookup

```

1 map<string,int> mp;              // sorted by key, O(log n) operations
2 mp["apple"] = 5;
3 mp["banana"]++;                // creates key with value 0 first if absent, then
    increments
4 if (mp.count("apple")) { /* key exists */ }
5 for (auto &[key, val] : mp) {    // iterates in sorted key order
6     cout << key << " " << val << "\n";
7 }
8
9 unordered_map<string,int> ump;   // hash table, O(1) average, O(n) worst case
10                                // faster on average but NOT sorted, and can be attacked
11                                // (adversarial TLE) on Codeforces -- prefer map if
    unsure
12 unordered_set<int> uset;        // O(1) average membership test
```

3.5 Stack, Queue, Deque, Priority Queue

```

1 stack<int> st;
2 st.push(1); st.top(); st.pop(); // LIFO
3
4 queue<int> q;
5 q.push(1); q.front(); q.pop();  // FIFO
6
7 deque<int> dq;                  // double-ended queue, O(1) push/pop both ends
8 dq.push_back(1); dq.push_front(2);
9 dq.pop_back(); dq.pop_front();
10
11 // priority_queue = binary heap. Default = MAX-heap (largest on top)
12 priority_queue<int> maxHeap;
13 maxHeap.push(5); maxHeap.top(); maxHeap.pop();
14
15 // MIN-heap (very common in Dijkstra / Huffman / merge-k-lists problems)
16 priority_queue<int, vector<int>, greater<int>> minHeap;
17
18 // Custom comparator: heap of pairs {distance, node}, smallest distance first
19 priority_queue<pii, vector<pii>, greater<pii>> pq;
```

3.6 String Functions

```

1 string s = "hello world";
2 s.substr(0, 5);                 // "hello" -- substring(startIndex, length)
3 s.find("world");                // index of first match, or string::npos if not found
4 s + " again";                  // concatenation
5 s.length(); s.size();          // same thing
6 reverse(s.begin(), s.end());
7 to_string(123);                // int -> string
8 stoi("123"); stoll("123456789012"); // string -> int / long long
9 sort(s.begin(), s.end());      // sorts the characters
10
11 // splitting by a delimiter using stringstream
12 stringstream ss("a,b,c");
13 string token;
14 vector<string> tokens;
15 while (getline(ss, token, ',')) tokens.push_back(token);
```

3.7 Bitset – compact fixed-size bit array

3.8 Other Handy ‘<algorithm>’ Functions

4 Sorting, Searching & Two-Pointer Techniques

Prepared for competition use

```

5         int sum = v[l] + v[r];
6         if (sum == target) return true;
7         else if (sum < target) l++;
8         else r--;
9     }
10    return false;
11 }

```

4.4 Sliding Window (fixed and variable size)

```

1 // Maximum sum of any window of size k
2 ll maxSumWindowK(vector<int> &v, int k) {
3     ll sum = 0;
4     for (int i = 0; i < k; i++) sum += v[i];
5     ll best = sum;
6     for (int i = k; i < sz(v); i++) {
7         sum += v[i] - v[i - k]; // slide window forward by 1
8         best = max(best, sum);
9     }
10    return best;
11 }
12
13 // Variable-size window: smallest subarray with sum >= target
14 int smallestSubarrayWithSum(vector<int> &v, int target) {
15     int n = sz(v), l = 0, best = INT_MAX;
16     ll sum = 0;
17     for (int r = 0; r < n; r++) {
18         sum += v[r];
19         while (sum >= target) { // shrink from the left while valid
20             best = min(best, r - l + 1);
21             sum -= v[l++];
22         }
23     }
24     return best == INT_MAX ? -1 : best;
25 }

```

5 Number Theory

5.1 GCD, LCM, Fast Exponentiation

```

1 ll gcd(ll a, ll b) { return b == 0 ? a : gcd(b, a % b); } // or just use __gcd(a,b)
2 ll lcm(ll a, ll b) { return a / gcd(a, b) * b; } // divide first to avoid
// overflow
3
4 // Fast (binary) exponentiation: a^b mod m in O(log b)
5 ll power(ll a, ll b, ll m) {
6     a %= m;
7     ll res = 1;
8     while (b > 0) {
9         if (b & 1) res = (res * a) % m; // if current bit is 1, multiply into result
10        a = (a * a) % m; // square the base
11        b >>= 1; // move to next bit
12    }
13    return res;
14 }

```

5.2 Modular Inverse (needed for modular division)

```

1 // Works when MOD is prime (Fermat's Little Theorem): a^(-1) = a^(MOD-2) mod MOD
2 ll modInverse(ll a, ll mod) { return power(a, mod - 2, mod); }
3
4 // (a / b) mod m == (a * modInverse(b, m)) mod m

```

5.3 Sieve of Eratosthenes – all primes up to N

```

1 const int MAXN = 1000001;
2 bool isComposite[MAXN]; // isComposite[i] == false and i>=2 means i is prime

```



```

3
4 void sieve() {
5     isComposite[0] = isComposite[1] = true;
6     for (int i = 2; (ll)i * i < MAXN; i++) {
7         if (!isComposite[i]) {
8             for (int j = i * i; j < MAXN; j += i)
9                 isComposite[j] = true;
10        }
11    }
12 }
13 // Total time: O(N log log N)

```

5.4 Prime Factorization & Divisors

```

1 // O(sqrt(n)) prime factorization
2 vector<pll> primeFactorize(ll n) {
3     vector<pll> factors; // {prime, exponent}
4     for (ll p = 2; p * p <= n; p++) {
5         if (n % p == 0) {
6             int exp = 0;
7             while (n % p == 0) { n /= p; exp++; }
8             factors.push_back({p, exp});
9         }
10    }
11    if (n > 1) factors.push_back({n, 1}); // remaining n is prime
12    return factors;
13 }
14
15 // O(sqrt(n)) to list all divisors
16 vector<ll> divisors(ll n) {
17     vector<ll> divs;
18     for (ll i = 1; i * i <= n; i++) {
19         if (n % i == 0) {
20             divs.push_back(i);
21             if (i != n / i) divs.push_back(n / i);
22         }
23     }
24     return divs;
25 }
26
27 // O(sqrt(n)) primality test
28 bool isPrime(ll n) {
29     if (n < 2) return false;
30     for (ll i = 2; i * i <= n; i++)
31         if (n % i == 0) return false;
32     return true;
33 }

```

6 Graph Algorithms

6.1 Representation & Traversal (BFS / DFS)

```

1 const int MAXV = 100005;
2 vector<int> adj[MAXV]; // adjacency list -- reset with adj[i].clear() between test
   cases
3 bool visited[MAXV];
4
5 void addEdge(int u, int v) { // undirected edge
6     adj[u].push_back(v);
7     adj[v].push_back(u);
8 }
9
10 // BFS -- shortest path in an UNWEIGHTED graph, level-by-level
11 vector<int> dist_(MAXV, -1);
12 void bfs(int src) {
13     fill(dist_.begin(), dist_.end(), -1);
14     queue<int> q;
15     q.push(src);

```

```

16     dist_[src] = 0;
17     while (!q.empty()) {
18         int u = q.front(); q.pop();
19         for (int v : adj[u]) {
20             if (dist_[v] == -1) {          // not visited yet
21                 dist_[v] = dist_[u] + 1;
22                 q.push(v);
23             }
24         }
25     }
26 }
27
28 // DFS -- explore as deep as possible; good for connectivity, cycle detection, components
29 void dfs(int u) {
30     visited[u] = true;
31     for (int v : adj[u]) {
32         if (!visited[v]) dfs(v);
33     }
34 }

```

6.2 Dijkstra – shortest path, weighted, no negative edges

```

1 vector<pii> wadj[MAXV];    // wadj[u] = list of {neighbour, weight}
2
3 vector<ll> dijkstra(int src, int n) {
4     vector<ll> dist(n + 1, INF);
5     priority_queue<pll, vector<pll>, greater<pll>> pq; // min-heap of {dist, node}
6     dist[src] = 0;
7     pq.push({0, src});
8     while (!pq.empty()) {
9         auto [d, u] = pq.top(); pq.pop();
10        if (d > dist[u]) continue; // outdated entry, skip (lazy deletion)
11        for (auto &[v, w] : wadj[u]) {
12            if (dist[u] + w < dist[v]) {
13                dist[v] = dist[u] + w;
14                pq.push({dist[v], v});
15            }
16        }
17    }
18    return dist; // O((V+E) log V)
19 }

```

6.3 Bellman-Ford – handles negative edges, detects negative cycles

```

1 struct Edge { int u, v; ll w; };
2 vector<Edge> edges;
3
4 vector<ll> bellmanFord(int src, int n, bool &hasNegCycle) {
5     vector<ll> dist(n + 1, INF);
6     dist[src] = 0;
7     for (int i = 1; i <= n - 1; i++) { // relax all edges (n-1) times
8         for (auto &e : edges) {
9             if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v])
10                dist[e.v] = dist[e.u] + e.w;
11         }
12    }
13    hasNegCycle = false;
14    for (auto &e : edges) { // one more pass detects negative cycles
15        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) hasNegCycle = true;
16    }
17    return dist; // O(V * E)
18 }

```

6.4 Floyd-Warshall – all-pairs shortest path

```

1 ll d[505][505]; // d[i][j] = shortest distance i -> j; init to INF, d[i][i] = 0
2
3 void floydWarshall(int n) {

```

```

4   for (int k = 1; k <= n; k++)           // intermediate node
5       for (int i = 1; i <= n; i++)
6           for (int j = 1; j <= n; j++)
7               if (d[i][k] < INF && d[k][j] < INF)
8                   d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
9 }
10 // O(V^3) -- only feasible for V <= ~500

```

6.5 Topological Sort (Kahn's Algorithm, BFS-based)

```

1 // Works on a Directed Acyclic Graph (DAG). Orders nodes so every edge u->v has u before v.
2 vector<int> topoSort(int n) {
3     vector<int> indeg(n + 1, 0), order;
4     for (int u = 1; u <= n; u++)
5         for (int v : adj[u]) indeg[v]++;
6
7     queue<int> q;
8     for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
9
10    while (!q.empty()) {
11        int u = q.front(); q.pop();
12        order.push_back(u);
13        for (int v : adj[u]) {
14            if (--indeg[v] == 0) q.push(v);
15        }
16    }
17    // if order.size() < n, the graph has a cycle -- no valid topological order
18    return order;
19 }

```

6.6 Disjoint Set Union (DSU / Union-Find)

```

1 struct DSU {
2     vector<int> parent, rnk;
3     DSU(int n) : parent(n + 1), rnk(n + 1, 0) {
4         iota(parent.begin(), parent.end(), 0); // parent[i] = i initially
5     }
6     int find(int x) {
7         if (parent[x] != x) parent[x] = find(parent[x]); // path compression
8         return parent[x];
9     }
10    bool unite(int a, int b) { // union by rank
11        a = find(a); b = find(b);
12        if (a == b) return false; // already in same set
13        if (rnk[a] < rnk[b]) swap(a, b);
14        parent[b] = a;
15        if (rnk[a] == rnk[b]) rnk[a]++;
16        return true;
17    }
18 };
19 // Both find() and unite() are ~O(alpha(n)), practically O(1)

```

6.7 Minimum Spanning Tree – Kruskal's Algorithm

```

1 struct WEdge { ll w; int u, v; };
2 bool cmpEdge(const WEdge &a, const WEdge &b) { return a.w < b.w; }
3
4 ll kruskalMST(vector<WEdge> &edges, int n) {
5     sort(edges.begin(), edges.end(), cmpEdge);
6     DSU dsu(n);
7     ll totalWeight = 0;
8     for (auto &e : edges) {
9         if (dsu.unite(e.u, e.v)) { // adding this edge doesn't create a cycle
10             totalWeight += e.w;
11         }
12     }
13     return totalWeight; // O(E log E)
14 }

```

6.8 Minimum Spanning Tree – Prim’s Algorithm

```

1 11 primMST(int n) { // uses wadj[] from the Dijkstra section
2     vector<bool> inMST(n + 1, false);
3     priority_queue<pll, vector<pll>, greater<pll>> pq; // {weight, node}
4     pq.push({0, 1}); // start from node 1
5     ll total = 0;
6     int count = 0;
7     while (!pq.empty() && count < n) {
8         auto [w, u] = pq.top(); pq.pop();
9         if (inMST[u]) continue;
10        inMST[u] = true;
11        total += w;
12        count++;
13        for (auto &[v, wt] : wadj[u]) {
14            if (!inMST[v]) pq.push({wt, v});
15        }
16    }
17    return total; // O(E log V)
18 }

```

7 Trees & Range Query Structures

7.1 Fenwick Tree / Binary Indexed Tree (BIT)

```

1 // Supports point update + prefix sum query, both in O(log n). 1-indexed.
2 struct Fenwick {
3     vector<ll> tree;
4     int n;
5     Fenwick(int n) : n(n), tree(n + 1, 0) {}
6
7     void update(int i, ll delta) { // add 'delta' to position i
8         for (; i <= n; i += i & (-i))
9             tree[i] += delta;
10    }
11    ll query(int i) { // prefix sum of [1..i]
12        ll sum = 0;
13        for (; i > 0; i -= i & (-i))
14            sum += tree[i];
15        return sum;
16    }
17    ll rangeQuery(int l, int r) { // sum of [l..r]
18        return query(r) - query(l - 1);
19    }
20 };

```

7.2 Segment Tree – range sum, point update

```

1 // 0-indexed array of size n. Supports range sum query and point update in O(log n).
2 struct SegTree {
3     int n;
4     vector<ll> tree;
5     SegTree(int n) : n(n), tree(4 * n, 0) {}
6
7     void build(vector<ll> &arr, int node, int l, int r) {
8         if (l == r) { tree[node] = arr[l]; return; }
9         int mid = (l + r) / 2;
10        build(arr, 2*node, l, mid);
11        build(arr, 2*node+1, mid+1, r);
12        tree[node] = tree[2*node] + tree[2*node+1];
13    }
14    void update(int node, int l, int r, int idx, ll val) {
15        if (l == r) { tree[node] = val; return; }
16        int mid = (l + r) / 2;
17        if (idx <= mid) update(2*node, l, mid, idx, val);
18        else update(2*node+1, mid+1, r, idx, val);
19        tree[node] = tree[2*node] + tree[2*node+1];
20    }

```

```

21 ll query(int node, int l, int r, int ql, int qr) { // sum of [ql, qr]
22     if (qr < l || r < ql) return 0; // no overlap
23     if (ql <= l && r <= qr) return tree[node]; // total overlap
24     int mid = (l + r) / 2; // partial overlap
25     return query(2*node, l, mid, ql, qr) + query(2*node+1, mid+1, r, ql, qr);
26 }
27 };
28 // Usage: SegTree st(n); st.build(arr, 1, 0, n-1);
29 // st.update(1, 0, n-1, idx, val);
30 // ll sum = st.query(1, 0, n-1, ql, qr);
31 // For range MIN/MAX instead of SUM, just replace the '+' with min()/max().

```

7.3 Sparse Table – $O(1)$ range minimum query (static array, no updates)

```

1 const int LOG = 20;
2 int sparse[LOG][MAXV];
3 int arr[MAXV];
4
5 void buildSparse(int n) {
6     for (int i = 0; i < n; i++) sparse[0][i] = arr[i];
7     for (int j = 1; (1 << j) <= n; j++)
8         for (int i = 0; i + (1 << j) - 1 < n; i++)
9             sparse[j][i] = min(sparse[j-1][i], sparse[j-1][i + (1 << (j-1))]);
10 }
11 int queryMin(int l, int r) { // inclusive range [l, r], 0-indexed
12     int j = log2(r - l + 1);
13     return min(sparse[j][l], sparse[j][r - (1 << j) + 1]);
14 }
15 // Build:  $O(n \log n)$ . Query:  $O(1)$ . Great when there are NO updates.

```

7.4 Binary Lifting – LCA (Lowest Common Ancestor) in $O(\log n)$ per query

```

1 int up[LOG][MAXV]; // up[j][v] = 2^j-th ancestor of v
2 int depth[MAXV];
3
4 void dfsLCA(int u, int parent) {
5     up[0][u] = parent;
6     for (int j = 1; j < LOG; j++)
7         up[j][u] = up[j-1][ up[j-1][u] ];
8     for (int v : adj[u]) {
9         if (v != parent) {
10             depth[v] = depth[u] + 1;
11             dfsLCA(v, u);
12         }
13     }
14 }
15 int lca(int u, int v) {
16     if (depth[u] < depth[v]) swap(u, v);
17     int diff = depth[u] - depth[v];
18     for (int j = 0; j < LOG; j++)
19         if (diff & (1 << j)) u = up[j][u]; // lift u to v's depth
20     if (u == v) return u;
21     for (int j = LOG - 1; j >= 0; j--) {
22         if (up[j][u] != up[j][v]) { u = up[j][u]; v = up[j][v]; }
23     }
24     return up[0][u];
25 }
26 // Call dfsLCA(root, root) once first (root's own parent = root, up[j][root] stays root).

```

8 Dynamic Programming

8.1 0/1 Knapsack

```

1 // n items, capacity W. weight[i], value[i]. Maximise total value without exceeding W.
2 int knapsack(int n, int W, vector<int> &weight, vector<int> &value) {
3     vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
4     for (int i = 1; i <= n; i++) {

```

```

5     for (int w = 0; w <= W; w++) {
6         dp[i][w] = dp[i-1][w];           // don't take item i
7         if (weight[i-1] <= w)
8             dp[i][w] = max(dp[i][w], dp[i-1][w - weight[i-1]] + value[i-1]); // take it
9     }
10 }
11 return dp[n][W];                       // O(n * W)
12 }
13 // Space-optimised 1D version (iterate w DESCENDING to avoid reusing an item):
14 int knapsack1D(int n, int W, vector<int> &weight, vector<int> &value) {
15     vector<int> dp(W + 1, 0);
16     for (int i = 0; i < n; i++)
17         for (int w = W; w >= weight[i]; w--)
18             dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);
19     return dp[W];
20 }

```

8.2 Longest Increasing Subsequence (LIS)

```

1 // O(n log n) using binary search (patience sorting)
2 int lis(vector<int> &v) {
3     vector<int> tails;           // tails[i] = smallest possible tail of an
4                                 // increasing subsequence of length i+1
5     for (int x : v) {
6         auto it = lower_bound(tails.begin(), tails.end(), x);
7         if (it == tails.end()) tails.push_back(x);
8         else *it = x;
9     }
10    return sz(tails);
11 }
12 // For STRICTLY increasing use lower_bound; for non-decreasing use upper_bound.

```

8.3 Longest Common Subsequence (LCS)

```

1 int lcs(string &a, string &b) {
2     int n = sz(a), m = sz(b);
3     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
4     for (int i = 1; i <= n; i++) {
5         for (int j = 1; j <= m; j++) {
6             if (a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
7             else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
8         }
9     }
10    return dp[n][m];              // O(n * m)
11 }

```

8.4 Coin Change (minimum coins to make amount)

```

1 int coinChange(vector<int> &coins, int amount) {
2     vector<int> dp(amount + 1, IINF);
3     dp[0] = 0;
4     for (int a = 1; a <= amount; a++)
5         for (int c : coins)
6             if (c <= a && dp[a - c] != IINF)
7                 dp[a] = min(dp[a], dp[a - c] + 1);
8     return dp[amount] == IINF ? -1 : dp[amount]; // O(amount * numCoins)
9 }

```

8.5 DP General Tips

- Identify the **state** (what changes between subproblems) and the **transition** (how states relate).
- Always define the **base case** clearly first.
- If memory is tight, check if you only need the previous row/column – roll the array (see 1D knapsack above).
- Memoized recursion (top-down with a **map** or array cache) is often easier to write correctly under time pressure than a bottom-up table – use whichever your team codes faster.

9 String Algorithms

9.1 KMP (Knuth-Morris-Pratt) – pattern matching in $O(n + m)$

```

1 // Builds the "failure function" (longest proper prefix that is also a suffix)
2 vector<int> buildLPS(string &pat) {
3     int m = sz(pat);
4     vector<int> lps(m, 0);
5     int len = 0, i = 1;
6     while (i < m) {
7         if (pat[i] == pat[len]) {
8             lps[i++] = ++len;
9         } else if (len > 0) {
10            len = lps[len - 1];
11        } else {
12            lps[i++] = 0;
13        }
14    }
15    return lps;
16 }
17
18 vector<int> kmpSearch(string &text, string &pat) {
19     vector<int> lps = buildLPS(pat), matches;
20     int n = sz(text), m = sz(pat), i = 0, j = 0;
21     while (i < n) {
22         if (text[i] == pat[j]) { i++; j++; }
23         if (j == m) { // found a full match ending at i-1
24             matches.push_back(i - j);
25             j = lps[j - 1];
26         } else if (i < n && text[i] != pat[j]) {
27             if (j > 0) j = lps[j - 1];
28             else i++;
29         }
30     }
31     return matches; // list of starting indices of matches
32 }

```

9.2 Z-Function – another $O(n)$ string matching / prefix tool

```

1 // z[i] = length of the longest substring starting at i that matches a prefix of s
2 vector<int> zFunction(string &s) {
3     int n = sz(s);
4     vector<int> z(n, 0);
5     int l = 0, r = 0;
6     for (int i = 1; i < n; i++) {
7         if (i < r) z[i] = min(r - i, z[i - 1]);
8         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
9         if (i + z[i] > r) { l = i; r = i + z[i]; }
10    }
11    return z;
12 }
13 // To find pattern P in text T: build z-function of (P + '#' + T), matches are where z[i]
    == |P|.

```

9.3 Trie – prefix tree for fast string prefix queries

```

1 struct TrieNode {
2     TrieNode* child[26] = {};
3     bool isEnd = false;
4 };
5
6 struct Trie {
7     TrieNode* root = new TrieNode();
8     void insert(string &word) {
9         TrieNode* cur = root;
10        for (char c : word) {
11            int idx = c - 'a';
12            if (!cur->child[idx]) cur->child[idx] = new TrieNode();
13            cur = cur->child[idx];
14        }

```

```

15     cur->isEnd = true;
16 }
17 bool search(string &word) { // exact word exists?
18     TrieNode* cur = root;
19     for (char c : word) {
20         int idx = c - 'a';
21         if (!cur->child[idx]) return false;
22         cur = cur->child[idx];
23     }
24     return cur->isEnd;
25 }
26 bool startsWith(string &prefix) { // any word starts with prefix?
27     TrieNode* cur = root;
28     for (char c : prefix) {
29         int idx = c - 'a';
30         if (!cur->child[idx]) return false;
31         cur = cur->child[idx];
32     }
33     return true;
34 }
35 };

```

9.4 Polynomial String Hashing – $O(1)$ substring comparison after $O(n)$ precompute

```

1 const ll HBASE = 131, HMOD = 1e9 + 7;
2 vector<ll> prefixHash, powBase;
3
4 void buildHash(string &s) {
5     int n = sz(s);
6     prefixHash.assign(n + 1, 0);
7     powBase.assign(n + 1, 1);
8     for (int i = 0; i < n; i++) {
9         prefixHash[i+1] = (prefixHash[i] * HBASE + s[i]) % HMOD;
10        powBase[i+1] = (powBase[i] * HBASE) % HMOD;
11    }
12 }
13 // Hash of substring s[l..r] (0-indexed, inclusive)
14 ll getHash(int l, int r) {
15     ll h = (prefixHash[r+1] - prefixHash[l] * powBase[r-l+1]) % HMOD;
16     return (h % HMOD + HMOD) % HMOD; // ensure non-negative
17 }
18 // Compare getHash(l1,r1) == getHash(l2,r2) to check if two substrings are (very likely)
19 // equal.
20 // Use two different (base, mod) pairs and combine if you need to defend against hash
21 // collisions.

```

10 Basic Computational Geometry

```

1 struct Point { ll x, y; };
2
3 // Cross product of vectors OA and OB. Sign tells turning direction.
4 ll cross(Point O, Point A, Point B) {
5     return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) * (B.x - O.x);
6 }
7 // cross > 0 : counter-clockwise turn (B is left of OA)
8 // cross < 0 : clockwise turn (B is right of OA)
9 // cross == 0: O, A, B are collinear
10
11 double dist(Point a, Point b) {
12     return sqrt((double)(a.x-b.x)*(a.x-b.x) + (double)(a.y-b.y)*(a.y-b.y));
13 }
14
15 // Convex Hull -- Andrew's monotone chain,  $O(n \log n)$ 
16 vector<Point> convexHull(vector<Point> pts) {
17     int n = sz(pts), k = 0;
18     if (n <= 2) return pts;
19     sort(pts.begin(), pts.end(), [](Point a, Point b){
20         return a.x < b.x || (a.x == b.x && a.y < b.y);

```



```

21     });
22     vector<Point> hull(2 * n);
23     for (int i = 0; i < n; i++) { // build lower hull
24         while (k >= 2 && cross(hull[k-2], hull[k-1], pts[i]) <= 0) k--;
25         hull[k++] = pts[i];
26     }
27     for (int i = n - 2, lo = k + 1; i >= 0; i--) { // build upper hull
28         while (k >= lo && cross(hull[k-2], hull[k-1], pts[i]) <= 0) k--;
29         hull[k++] = pts[i];
30     }
31     hull.resize(k - 1);
32     return hull;
33 }

```

11 Bit Manipulation

```

1  int x = 13; // binary: 1101
2  x & 1; // check last bit (is x odd?)
3  x >> 1; // divide by 2 (integer division)
4  x << 1; // multiply by 2
5  x & (x - 1); // clear the lowest set bit
6  x & (-x); // isolate the lowest set bit
7  __builtin_popcount(x); // count of set bits (int)
8  __builtin_popcountll(x); // count of set bits (long long)
9  __builtin_ctz(x); // count trailing zeros (x != 0)
10 __builtin_clz(x); // count leading zeros (x != 0)
11
12 // Iterate over all subsets of a bitmask 'mask' (useful for bitmask DP)
13 for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
14     // process 'sub'
15 }
16
17 // Iterate over all 2^n subsets of an n-element set
18 for (int mask = 0; mask < (1 << n); mask++) {
19     for (int i = 0; i < n; i++) {
20         if (mask & (1 << i)) { /* element i is in this subset */ }
21     }
22 }

```

12 More Number Theory: Extended Euclid & Combinatorics

12.1 Extended Euclidean Algorithm

```

1 // Finds x, y such that a*x + b*y = gcd(a, b). Used for modular inverse when MOD is not
   prime,
2 // and for solving linear Diophantine equations.
3 ll extendedGCD(ll a, ll b, ll &x, ll &y) {
4     if (b == 0) { x = 1; y = 0; return a; }
5     ll x1, y1;
6     ll g = extendedGCD(b, a % b, x1, y1);
7     x = y1;
8     y = x1 - (a / b) * y1;
9     return g; // returns gcd(a, b)
10 }
11
12 // Modular inverse using extended Euclid (works for ANY mod, as long as gcd(a, mod) == 1)
13 ll modInverseExt(ll a, ll mod) {
14     ll x, y;
15     ll g = extendedGCD(a, mod, x, y);
16     if (g != 1) return -1; // inverse doesn't exist
17     return ((x % mod) + mod) % mod;
18 }

```

12.2 Combinatorics – nCr mod p, Factorials, Pascal's Triangle

```

1 const int NMAX = 200005;
2 ll fact[NMAX], invFact[NMAX];

```

```

3
4 void precomputeFactorials(ll mod) {                               // call once, O(NMAX log mod)
5     fact[0] = 1;
6     for (int i = 1; i < NMAX; i++) fact[i] = (fact[i-1] * i) % mod;
7     invFact[NMAX-1] = modInverse(fact[NMAX-1], mod);
8     for (int i = NMAX - 2; i >= 0; i--)
9         invFact[i] = (invFact[i+1] * (i+1)) % mod;
10 }
11 ll nCr(int n, int r, ll mod) {                                    // O(1) after precompute
12     if (r < 0 || r > n) return 0;
13     return fact[n] * invFact[r] % mod * invFact[n-r] % mod;
14 }
15
16 // Pascal's triangle -- simpler alternative for small n (no modular inverse needed)
17 ll pascal[1005][1005];
18 void buildPascal(int n) {
19     for (int i = 0; i <= n; i++) {
20         pascal[i][0] = 1;
21         for (int j = 1; j <= i; j++)
22             pascal[i][j] = (pascal[i-1][j-1] + pascal[i-1][j]) % MOD;
23     }
24 }

```

13 Advanced Graphs

13.1 0-1 BFS – shortest path when edge weights are only 0 or 1

```

1 // Faster than Dijkstra for this special case: O(V + E) using a deque instead of a heap.
2 vector<pii> zoAdj[MAXV];    // {neighbour, weight(0 or 1)}
3
4 vector<int> zeroOneBFS(int src, int n) {
5     vector<int> dist(n + 1, IINF);
6     deque<int> dq;
7     dist[src] = 0;
8     dq.push_front(src);
9     while (!dq.empty()) {
10         int u = dq.front(); dq.pop_front();
11         for (auto &[v, w] : zoAdj[u]) {
12             if (dist[u] + w < dist[v]) {
13                 dist[v] = dist[u] + w;
14                 if (w == 0) dq.push_front(v);    // 0-weight edge: push to front
15                 else dq.push_back(v);           // 1-weight edge: push to back
16             }
17         }
18     }
19     return dist;
20 }

```

13.2 Strongly Connected Components – Kosaraju's Algorithm

```

1 // Groups nodes of a DIRECTED graph into maximal sets where every node can reach every
2 // other node in the same set. O(V + E).
3 vector<int> kAdj[MAXV], kAdjRev[MAXV];
4 vector<int> order_, component;
5 bool visited_[MAXV];
6
7 void dfs1(int u) {                                                // first pass: fill finish-time order
8     visited_[u] = true;
9     for (int v : kAdj[u]) if (!visited_[v]) dfs1(v);
10    order_.push_back(u);
11 }
12
13 void dfs2(int u, int compId) {                                     // second pass: assign component ids
14     component[u] = compId;
15     for (int v : kAdjRev[u]) if (component[v] == -1) dfs2(v, compId);
16 }
17
18 int kosaraju(int n) {
19     order_.clear();
20     fill(visited_, visited_ + n + 1, false);

```

```

19     for (int i = 1; i <= n; i++) if (!visited_[i]) dfs1(i);
20
21     component.assign(n + 1, -1);
22     int numComponents = 0;
23     for (int i = n - 1; i >= 0; i--) {
24         int u = order_[i];
25         if (component[u] == -1) dfs2(u, numComponents++);
26     }
27     return numComponents;
28     // Build kAdjRev by adding every edge (u,v) from kAdj as (v,u) into kAdjRev beforehand.
29 }

```

13.3 Segment Tree with Lazy Propagation – range update, range sum query

```

1 // Supports "add val to every element in [l,r]" AND "sum of [l,r]", both in O(log n).
2 struct LazySegTree {
3     int n;
4     vector<ll> tree, lazy;
5     LazySegTree(int n) : n(n), tree(4*n, 0), lazy(4*n, 0) {}
6
7     void push(int node, int l, int r) {
8         if (lazy[node] == 0) return;
9         tree[node] += (ll)(r - l + 1) * lazy[node];
10        if (l != r) { // propagate to children
11            lazy[2*node] += lazy[node];
12            lazy[2*node+1] += lazy[node];
13        }
14        lazy[node] = 0;
15    }
16    void update(int node, int l, int r, int ql, int qr, ll val) {
17        push(node, l, r);
18        if (qr < l || r < ql) return;
19        if (ql <= l && r <= qr) {
20            lazy[node] += val;
21            push(node, l, r);
22            return;
23        }
24        int mid = (l + r) / 2;
25        update(2*node, l, mid, ql, qr, val);
26        update(2*node+1, mid+1, r, ql, qr, val);
27        tree[node] = tree[2*node] + tree[2*node+1];
28    }
29    ll query(int node, int l, int r, int ql, int qr) {
30        push(node, l, r);
31        if (qr < l || r < ql) return 0;
32        if (ql <= l && r <= qr) return tree[node];
33        int mid = (l + r) / 2;
34        return query(2*node, l, mid, ql, qr) + query(2*node+1, mid+1, r, ql, qr);
35    }
36 };

```

14 Bitmask DP, Digit DP & Matrix Exponentiation

14.1 Bitmask DP – Travelling Salesman Problem (TSP)

```

1 // Minimum cost to visit all n cities starting at city 0. n <= ~18 (2^n states).
2 ll costMat[20][20];
3 ll memo[1 << 18][18];
4
5 ll tsp(int mask, int pos, int n) {
6     if (mask == (1 << n) - 1) return costMat[pos][0]; // all visited, return to start
7     if (memo[mask][pos] != -1) return memo[mask][pos];
8
9     ll best = INF;
10    for (int city = 0; city < n; city++) {
11        if (!(mask & (1 << city))) { // city not yet visited
12            ll newCost = costMat[pos][city] + tsp(mask | (1 << city), city, n);
13            best = min(best, newCost);

```

```

14     }
15 }
16 return memo[mask][pos] = best;
17 }
18 // Init memo to -1, call tsp(1, 0, n). Complexity:  $O(2^n * n^2)$ 

```

14.2 Digit DP – count numbers in [0, N] satisfying a digit property

```

1 // Example: count numbers <= N whose digit sum equals a target.
2 string numStr;
3 ll digitMemo[20][200];
4 bool digitVis[20][200];
5
6 ll digitDP(int pos, int sumSoFar, bool tight) {
7     if (pos == sz(numStr)) return sumSoFar == 0 ? 1 : 0; // reached end, check condition
8     if (!tight && digitVis[pos][sumSoFar]) return digitMemo[pos][sumSoFar];
9
10    int limit = tight ? (numStr[pos] - '0') : 9;
11    ll ways = 0;
12    for (int d = 0; d <= limit; d++) {
13        if (d <= sumSoFar) // example condition
14            ways += digitDP(pos + 1, sumSoFar - d, tight && (d == limit));
15    }
16    if (!tight) { digitVis[pos][sumSoFar] = true; digitMemo[pos][sumSoFar] = ways; }
17    return ways;
18 }
19 // Call: numStr = to_string(N); memset(digitVis, false, sizeof digitVis);
20 //      digitDP(0, targetSum, true);

```

14.3 Matrix Exponentiation – $O(k^3 \log n)$ for linear recurrences (e.g. Fibonacci)

```

1 typedef vector<vector<ll>> Matrix;
2
3 Matrix multiply(Matrix &A, Matrix &B, ll mod) {
4     int n = sz(A), m = sz(B[0]), k = sz(B);
5     Matrix C(n, vector<ll>(m, 0));
6     for (int i = 0; i < n; i++)
7         for (int j = 0; j < m; j++)
8             for (int t = 0; t < k; t++)
9                 C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % mod;
10    return C;
11 }
12 Matrix matPower(Matrix base, ll exp, ll mod) {
13     int n = sz(base);
14     Matrix result(n, vector<ll>(n, 0));
15     for (int i = 0; i < n; i++) result[i][i] = 1; // identity matrix
16     while (exp > 0) {
17         if (exp & 1) result = multiply(result, base, mod);
18         base = multiply(base, base, mod);
19         exp >>= 1;
20     }
21     return result;
22 }
23 // Fibonacci example:  $[[F(n+1)], [F(n)]] = [[1, 1], [1, 0]]^n * [[F(1)], [F(0)]]$ 
24 ll fibonacci(ll n, ll mod) {
25     if (n == 0) return 0;
26     Matrix base = {{1, 1}, {1, 0}};
27     Matrix res = matPower(base, n - 1, mod);
28     return res[0][0];
29 }

```

15 Recursion & Backtracking

15.1 N-Queens – classic backtracking template

```

1 int n_, solutions = 0;
2 vector<int> col_; // col_[row] = column of the queen placed in that row

```

```

3
4 bool isSafe(int row, int col) {
5     for (int r = 0; r < row; r++) {
6         if (col_[r] == col) return false;           // same column
7         if (abs(col_[r] - col) == abs(r - row)) return false; // same diagonal
8     }
9     return true;
10 }
11 void solveNQueens(int row) {
12     if (row == n_) { solutions++; return; }         // found a valid placement
13     for (int c = 0; c < n_; c++) {
14         if (isSafe(row, c)) {
15             col_[row] = c;
16             solveNQueens(row + 1);
17             // no explicit "undo" needed here since col_[row] gets overwritten next
18             // iteration
19         }
20     }
21 // Call: col_.assign(n_, -1); solveNQueens(0);
22 // General backtracking pattern: choose -> recurse -> (undo if using shared mutable state)

```

15.2 Generating All Subsets / Permutations

```

1 // All subsets of a set of size n via recursion
2 vector<int> current;
3 void genSubsets(vector<int> &arr, int idx) {
4     if (idx == sz(arr)) {
5         // process 'current' here (this is one subset)
6         return;
7     }
8     genSubsets(arr, idx + 1);           // exclude arr[idx]
9     current.push_back(arr[idx]);
10    genSubsets(arr, idx + 1);           // include arr[idx]
11    current.pop_back();                 // backtrack
12 }
13
14 // All permutations using STL (simplest approach for contests)
15 vector<int> v = {1, 2, 3};
16 sort(v.begin(), v.end());             // must start sorted to get ALL permutations
17 do {
18     // process this permutation of v
19 } while (next_permutation(v.begin(), v.end()));

```

16 Bipartite Graph Check & Graph Colouring

```

1 // A graph is bipartite iff it can be 2-coloured such that no edge connects same-coloured
2 // nodes -- equivalently, iff it has no odd-length cycle. O(V + E) via BFS.
3 int color_[MAXV]; // -1 = uncoloured, 0/1 = the two colours
4
5 bool isBipartite(int n) {
6     fill(color_, color_ + n + 1, -1);
7     for (int start = 1; start <= n; start++) {
8         if (color_[start] != -1) continue;
9         color_[start] = 0;
10        queue<int> q; q.push(start);
11        while (!q.empty()) {
12            int u = q.front(); q.pop();
13            for (int v : adj[u]) {
14                if (color_[v] == -1) {
15                    color_[v] = 1 - color_[u];
16                    q.push(v);
17                } else if (color_[v] == color_[u]) {
18                    return false; // conflict -- not bipartite
19                }
20            }
21        }
22    }
23 }

```

```

23     return true;
24 }

```

17 STL Container Complexity Summary

Container	Access / Search	Insert / Erase
vector	$O(1)$ by index	$O(1)$ amortised at end, $O(n)$ elsewhere
deque	$O(1)$ by index	$O(1)$ at both ends
set / map	$O(\log n)$	$O(\log n)$
unordered_set / unordered_map	$O(1)$ average, $O(n)$ worst	$O(1)$ average
priority_queue	$O(1)$ top	$O(\log n)$ push/pop
stack / queue	$O(1)$ top/front	$O(1)$ push/pop

Rule of thumb: start with `vector`, `set`, and `map` – they are predictable. Switch to the `unordered_*` versions only when you need the extra average-case speed and don't need sorted order.

18 Prefix Sums & Interval Techniques

18.1 1D Prefix Sum – $O(1)$ range sum after $O(n)$ precompute

```

1 vector<ll> prefix;
2 void buildPrefix(vector<int> &v) {
3     int n = sz(v);
4     prefix.assign(n + 1, 0);
5     for (int i = 0; i < n; i++) prefix[i+1] = prefix[i] + v[i];
6 }
7 ll rangeSum(int l, int r) {                // sum of v[l..r], 0-indexed inclusive
8     return prefix[r+1] - prefix[l];
9 }

```

18.2 2D Prefix Sum – $O(1)$ rectangle sum queries

```

1 ll prefix2D[1005][1005];                // 1-indexed grid, size (n+1) x (m+1)
2 void build2D(vector<vector<int>> &grid, int n, int m) {
3     for (int i = 1; i <= n; i++)
4         for (int j = 1; j <= m; j++)
5             prefix2D[i][j] = grid[i-1][j-1] + prefix2D[i-1][j]
6                 + prefix2D[i][j-1] - prefix2D[i-1][j-1];
7 }
8 // Sum of rectangle with corners (r1,c1) top-left and (r2,c2) bottom-right, 1-indexed
9 ll rectSum(int r1, int c1, int r2, int c2) {
10     return prefix2D[r2][c2] - prefix2D[r1-1][c2] - prefix2D[r2][c1-1] + prefix2D[r1-1][c1-1];
11 }

```

18.3 Difference Array – $O(1)$ range update, $O(n)$ to rebuild

```

1 // Useful when you have MANY range-add updates and only need the final array once.
2 vector<ll> diff;
3 void rangeAdd(int l, int r, ll val, int n) {    // add val to v[l..r], 0-indexed
4     diff[l] += val;
5     if (r + 1 < n) diff[r + 1] -= val;
6 }
7 vector<ll> applyDiff(int n) {
8     vector<ll> result(n);
9     ll running = 0;
10    for (int i = 0; i < n; i++) { running += diff[i]; result[i] = running; }
11    return result;
12 }

```

18.4 Merging Overlapping Intervals

```

1 vector<pii> mergeIntervals(vector<pii> intervals) {
2     sort(intervals.begin(), intervals.end());    // sort by start
3     vector<pii> merged;

```

```

4     for (auto &iv : intervals) {
5         if (!merged.empty() && iv.first <= merged.back().second) {
6             merged.back().second = max(merged.back().second, iv.second);    // overlap ->
                merge
7         } else {
8             merged.push_back(iv);
9         }
10    }
11    return merged;                                // O(n log n)
12 }

```

19 Modular Arithmetic Quick Reference

```

1 // Basic rules (MOD is usually 1e9+7 or 998244353, both prime):
2 // (a + b) % MOD          -- direct
3 // (a - b + MOD) % MOD    -- ALWAYS add MOD before taking % to avoid negative results
4 // (a * b) % MOD          -- careful: a*b can overflow 'int', use 'long long'
5 // (a / b) % MOD = (a * modInverse(b, MOD)) % MOD -- NEVER divide directly under a modulus
6
7 // Common competitive programming moduli:
8 // 1e9+7 = 1000000007      (prime, most common)
9 // 998244353              (prime, "NTT-friendly", used in FFT/NTT problems)
10
11 // Watch out: MOD - 1 tricks show up in Fermat's little theorem (a^(MOD-1) = 1 mod MOD)

```

20 Complexity Cheat Sheet

20.1 Time Limit Rule of Thumb

Most judges give 1–2 seconds and allow roughly 10^8 – 10^9 simple operations per second. Use this table to pick a feasible algorithm once you know n :

Input size n	Complexity you can usually afford
$n \leq 10$	$O(n!)$, $O(2^n \cdot n)$ – brute force permutations
$n \leq 20$	$O(2^n)$ – bitmask DP, subset enumeration
$n \leq 500$	$O(n^3)$ – Floyd-Warshall, simple 3D DP
$n \leq 5000$	$O(n^2)$ – simple DP, nested loops
$n \leq 10^5$	$O(n \log n)$ – sorting, segment tree, Dijkstra with heap
$n \leq 10^6$ – 10^7	$O(n)$ or $O(n \log \log n)$ – sieve, two pointers, prefix sums
$n \leq 10^9$	$O(\log n)$ or $O(\sqrt{n})$ – binary search, fast power, factorization

20.2 Common Algorithm Complexities

Algorithm	Complexity
Sorting (sort())	$O(n \log n)$
Binary search	$O(\log n)$
BFS / DFS	$O(V + E)$
Dijkstra (with heap)	$O((V + E) \log V)$
Bellman-Ford	$O(V \cdot E)$
Floyd-Warshall	$O(V^3)$
Kruskal's MST	$O(E \log E)$
Segment Tree query/update	$O(\log n)$
Fenwick Tree query/update	$O(\log n)$
Sieve of Eratosthenes	$O(n \log \log n)$
KMP / Z-function	$O(n + m)$
0/1 Knapsack	$O(n \cdot W)$

21 Debugging & Submission Checklist

Run through this list *before* you submit, and again if you get Wrong Answer / Runtime Error / TLE.

21.1 Wrong Answer

- Re-read the problem statement – did you miss a constraint or an edge case (e.g. $n = 0$, single element, all values equal)?
- Check array indexing: is the problem 0-indexed or 1-indexed? Does your loop match?
- Integer overflow: does any intermediate value exceed `INT_MAX` ($\approx 2.1 \times 10^9$)? Switch to `long long`.
- Did you forget to reset global variables/arrays between multiple test cases?
- For floating point: are you comparing with an epsilon (e.g. `fabs(a-b) < 1e-9`) instead of `==`?

21.2 Runtime Error

- Array out of bounds – double check all array sizes are large enough (off-by-one).
- Division by zero.
- Recursive function with no/incorrect base case causing stack overflow.
- Accessing an empty container (e.g. `.top()` on an empty stack, `.front()` on empty queue).

21.3 Time Limit Exceeded

- Confirm fast I/O is enabled (Section 2).
- Check your complexity against n using the cheat sheet in Section 20.
- Avoid `unordered_map`/`unordered_set` of pairs/vectors without a custom hash – can degrade to $O(n)$ per operation.
- Look for accidental $O(n^2)$ behaviour hidden inside a loop (e.g. `v.erase()` or string concatenation inside a loop).
- Avoid passing large containers **by value** into functions – pass by reference (`&`).

21.4 General Testing Habits

- Test the given sample input/output exactly before submitting.
- Manually construct 1–2 small edge-case tests (minimum size, maximum size, all-equal values).
- Print intermediate variables to `cerr` (not `cout`) while debugging locally – `cerr` is unbuffered and separate from judged output.

22 Quick-Reference: Which Data Structure Do I Need?

You need to...	Use
Keep elements sorted with fast insert/erase	<code>set</code> / <code>multiset</code>
Fast key $\rightarrow$ value lookup, order doesn't matter	<code>unordered_map</code>
Fast key $\rightarrow$ value lookup, sorted order matters	<code>map</code>
Repeatedly get the min/max element	<code>priority_queue</code>
Process elements in insertion order (FIFO)	<code>queue</code>
Undo / matching (LIFO), e.g. balanced parentheses	<code>stack</code>
Range sum with point updates	Fenwick Tree
Range min/max/sum with point or range updates	Segment Tree
Range min query, no updates	Sparse Table
Connectivity / grouping queries (union-find)	DSU
Prefix-based string queries (autocomplete)	Trie
Shortest path, unweighted graph	BFS
Shortest path, weighted, non-negative	Dijkstra
Shortest path, negative weights allowed	Bellman-Ford
Shortest path, all pairs, small V	Floyd-Warshall

23 4-Week Practice Plan (Appendix)

Use this as a rough guide with your team leading up to the contest. Practice on Codeforces (Div. 2/3/4 rounds and the *problemset* filtered by tags) and `icpc.global` archived regionals.

Week	Focus
1	STL mastery (Section 3), sorting/searching/two-pointers (Section 4), simple brute force
2	Number theory (Section 5), basic graphs – BFS/DFS/Dijkstra (Section 6)
3	DSU, MST, segment/Fenwick trees (Sections 6–7), core DP patterns (Section 8)
4	Strings (Section 9), bitmask/digit DP (Section 11), full mock contests (5 problems, 3–5 hours, as a team)

23.1 Problem-Tag Checklist

Solve at least 5–10 problems per tag before the contest:

- **Implementation / Simulation** – careful reading, no tricky algorithm
- **Greedy** – prove to yourself why the greedy choice works before coding
- **Sorting + Two Pointers**
- **Binary Search** (including "binary search on the answer")
- **Graphs: BFS/DFS/Shortest Path**
- **Dynamic Programming: 1D and 2D**
- **Number Theory: GCD/Sieve/Modular Arithmetic**
- **Data Structures: DSU, Segment Tree, Fenwick Tree**
- **Strings: KMP/Hashing**
- **Ad hoc / Math** – these appear often in JRC-style sets and reward careful reasoning over memorised code

Good luck at your ICPC JRC contest!

Practice consistently, read every problem carefully, communicate as a team, and trust your templates.

Notes

Use this page during practice or the contest to jot down team-specific reminders, tricky edge cases you've been bitten by before, or extra snippets you want to add.
